The University of Azad Jammu & Kashmir, Muzaffarabad

Department of Software Engineering

Database Systems

Lab N0#04

Title: Database Normalization

Submitted to: Engr.Muhmmad Awais

Student Name: Feeza Kulsoom

Roll No: 2024-SE-03

Reg Number: 2024-UMDB-004738

Task 1: Functional Dependencies and Anomalies Analysis

1. Functional Dependencies

Functional dependencies describe the relationship between attributes in a relation. The following dependencies are identified:

FD1:
OrderID → OrderDate, CustID, CustName, CustEmail

Each OrderID uniquely determines the order date and customer details.

FD2:
CustID → CustName, CustEmail

Each customer ID uniquely determines the customer’ s name and email.

FD3:
BookID → BookTitle, Publisher, UnitPrice

Each book ID uniquely determines book-related information.

FD4:
(OrderID, BookID) → Qty

The combination of OrderID and BookID determines the quantity ordered.

2. Insertion Anomaly

Insertion anomaly occurs when certain data cannot be inserted into the database without the presence of other data.

In this case, a new book (e.g., B-4) cannot be added to the system unless it is associated with an existing order. This prevents storing independent book information.

3. Update Anomaly

Update anomaly occurs when the same data is repeated in multiple rows and must be updated in all places.

> For example, if the publisher "Pearson" changes its name, the update must be applied to all rows where it appears. Missing any row leads to data inconsistency.

4. Deletion Anomaly

Deletion anomaly occurs when deleting a record unintentionally removes additional important information.

> For instance, if order O-503 is deleted, all information related to the book "Networks" may also be lost, even if that book should remain in the system.

TASK 2: CONVERSION TO 1NF

OBJECTIVE:

The objective of this task is to convert the given unnormalized table into First Normal Form (1NF) by removing repeating groups and ensuring that all values are atomic.

PRIMARY KEY:

The primary key for this table is a composite key:
(OrderID, BookID)

REASON:

OrderID alone is not unique because one order can contain multiple books.
BookID alone is not unique because one book can appear in multiple orders.
The combination of OrderID and BookID uniquely identifies each record.

SQL CODE:

The complete SQL code for this task is also provided in the attached .sql file.

SCREENSHOT:

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)

```
SELECT * FROM OrderBook_1NF;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	OrderID	OrderDate	CustID	CustName	CustEmail	BookID	BookTitle	Publisher	UnitPrice	Qty
Edit Copy Delete	O-501	2026-04-02	C-11	Bilal	bilal@x.com	B-1	SQL Basics	Pearson	1200.00	1
Edit Copy Delete	O-501	2026-04-02	C-11	Bilal	bilal@x.com	B-2	Python 101	OReilly	1500.00	2
Edit Copy Delete	O-502	2026-04-03	C-12	Areeba	areeba@x.com	B-1	SQL Basics	Pearson	1200.00	3
Edit Copy Delete	O-503	2026-04-05	C-11	Bilal	bilal@x.com	B-2	Python 101	OReilly	1500.00	1
Edit Copy Delete	O-503	2026-04-05	C-11	Bilal	bilal@x.com	B-3	Networks	Pearson	1800.00	1

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

EXPLANATION:

The table is now in First Normal Form (1NF) because:

1. All values are atomic (each field contains a single value).
2. There are no repeating groups.
3. Each record is uniquely identified by the composite primary key (OrderID, BookID).
4. Each book in an order is stored as a separate row instead of multiple values in one field.

TASK 3: CONVERSION to 2NF

PARTIAL DEPENDENCIES IDENTIFIED:

In the 1NF table with composite primary key (OrderID, BookID), the following partial dependencies exist:

- OrderDate depends only on OrderID
- CustID depends only on OrderID
- CustName depends only on OrderID
- CustEmail depends only on OrderID
- BookTitle depends only on BookID
- Publisher depends only on BookID
- UnitPrice depends only on BookID
- Qty depends on both OrderID and BookID (Full Dependency)

SOLUTION (DECOMPOSITION):

To remove partial dependencies, the table is decomposed into three separate tables:

Table 1: Orders_2NF
Primary Key: OrderID
Contains: OrderDate, CustID, CustName, CustEmail

Table 2: Books_2NF
Primary Key: BookID
Contains: BookTitle, Publisher, UnitPrice

Table 3: OrderDetails_2NF
Primary Key: (OrderID, BookID)
Foreign Keys:

- OrderID references Orders_2NF
- BookID references Books_2NF
  Contains: Qty

SQL CODE:

The complete SQL code is provided in the attached .sql file.

SCREENSHOTS:

Screenshot 1: SELECT * FROM Orders_2NF;

Showing rows 0 - 2 (3 total, Query took 0.0004 seconds.)

```
SELECT * FROM Orders_2NF;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	OrderID	OrderDate	CustID	CustName	CustEmail
Edit Copy Delete	O-501	2026-04-02	C-11	Bilal	bilal@x.com
Edit Copy Delete	O-502	2026-04-03	C-12	Areeba	areeba@x.com
Edit Copy Delete	O-503	2026-04-05	C-11	Bilal	bilal@x.com

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Screenshot 2: SELECT * FROM Books_2NF;

Showing rows 0 - 2 (3 total, Query took 0.0002 seconds.)

```
SELECT * FROM Books_2NF;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

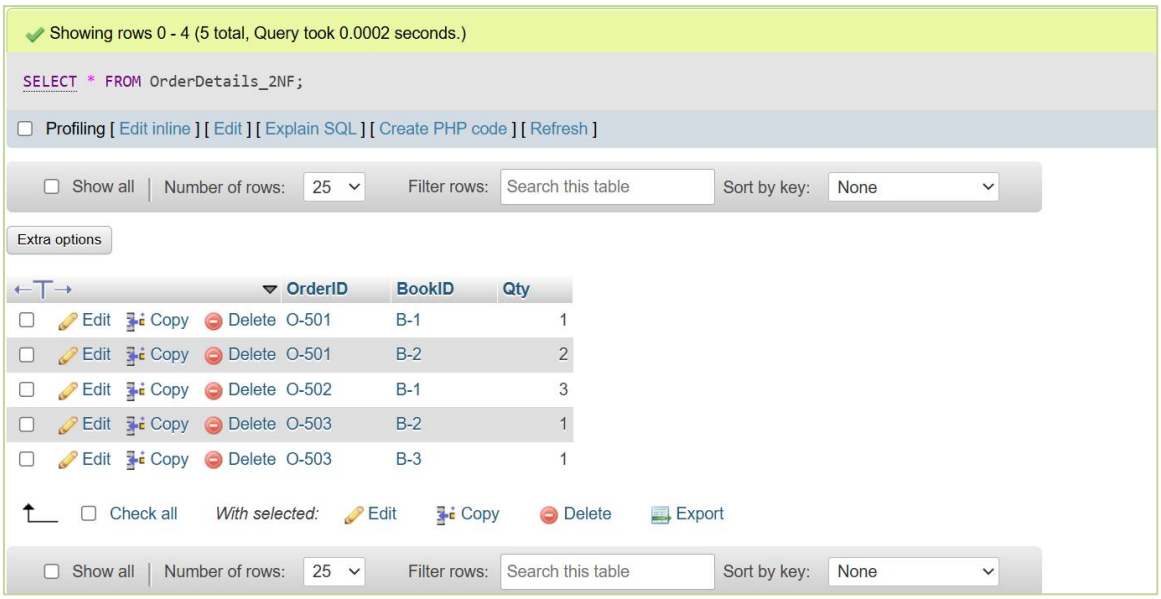
Extra options

	BookID	BookTitle	Publisher	UnitPrice
Edit Copy Delete	B-1	SQL Basics	Pearson	1200.00
Edit Copy Delete	B-2	Python 101	OReilly	1500.00
Edit Copy Delete	B-3	Networks	Pearson	1800.00

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Screenshot 3: SELECT * FROM OrderDetails_2NF;



EXPLANATION:

The table is now in Second Normal Form (2NF) because:

1. It is already in First Normal Form (1NF).
2. All partial dependencies have been removed.
3. Order-related data is stored separately in the Orders table.
4. Book-related data is stored separately in the Books table.
5. The relationship between orders and books is maintained in the OrderDetails table using a composite key.

TASK 4: CONVERSION TO 3NF

TRANSITIVE DEPENDENCIES IDENTIFIED:

In the 2NF table (Orders_2NF), the following transitive dependency exists:

OrderID → CustID → CustName, CustEmail

This means:
OrderID determines CustID
CustID determines CustName and CustEmail
Therefore, CustName and CustEmail transitively depend on OrderID

This creates redundancy because customer details are repeated for every order.

SOLUTION (DECOMPOSITION):

To remove transitive dependencies, the database is decomposed into four tables:

Table 1: Customers_3NF
Primary Key: CustID
Contains: CustName, CustEmail

Table 2: Orders_3NF
Primary Key: OrderID
Foreign Key: CustID references Customers_3NF
Contains: OrderDate

Table 3: Books_3NF
Primary Key: BookID
Contains: BookTitle, Publisher, UnitPrice

Table 4: OrderDetails_3NF
Primary Key: (OrderID, BookID)
Foreign Keys:

- OrderID references Orders_3NF
- BookID references Books_3NF
  Contains: Qty

SQL CODE:

The complete SQL code is provided in the attached .sql file.

SCREENSHOTS:

Screenshot 1: SELECT * FROM Customers_3NF;

Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.)

```
SELECT * FROM Customers_3NF;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	CustID	CustName	CustEmail
Edit Copy Delete	C-11	Bilal	bilal@x.com
Edit Copy Delete	C-12	Areeba	areeba@x.com

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Screenshot 2: SELECT * FROM Orders_3NF;

Showing rows 0 - 2 (3 total, Query took 0.0002 seconds.)

```
SELECT * FROM Orders_3NF;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	OrderID	OrderDate	CustID
Edit Copy Delete	O-501	2026-04-02	C-11
Edit Copy Delete	O-502	2026-04-03	C-12
Edit Copy Delete	O-503	2026-04-05	C-11

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Screenshot 3: SELECT * FROM Books_3NF;

Showing rows 0 - 2 (3 total, Query took 0.0002 seconds.)

```
SELECT * FROM Books_3NF;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	BookID	BookTitle	Publisher	UnitPrice
Edit Copy Delete	B-1	SQL Basics	Pearson	1200.00
Edit Copy Delete	B-2	Python 101	OReilly	1500.00
Edit Copy Delete	B-3	Networks	Pearson	1800.00

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Screenshot 4: SELECT * FROM OrderDetails_3NF;

Showing rows 0 - 4 (5 total, Query took 0.0002 seconds.)

```
SELECT * FROM OrderDetails_3NF;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	OrderID	BookID	Qty
Edit Copy Delete	O-501	B-1	1
Edit Copy Delete	O-501	B-2	2
Edit Copy Delete	O-502	B-1	3
Edit Copy Delete	O-503	B-2	1
Edit Copy Delete	O-503	B-3	1

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

EXPLANATION:

The database is now in Third Normal Form (3NF) because:

1. It is already in Second Normal Form (2NF).
2. All transitive dependencies have been removed.
3. Customer information is stored separately in the Customers table.
4. If a customer's details change, only one row needs to be updated.
5. No non-key attribute depends on another non-key attribute.

COMPARISON WITH 2NF:

In 2NF:
Customer name and email were repeated for each order.

In 3NF:
Customer details are stored only once in the Customers table, eliminating redundancy and update anomalies.

TASK 5: Verification Queries

PURPOSE:

These queries verify that the 3NF normalization is correct by:
1. Reproducing the original report (one row per book purchased)
2. Generating a customer spending report

QUERY 1: Reproduce Original Report

This query joins all 4 tables (Customers, Orders, Books, OrderDetails) to recreate exactly the same information that was in the original unnormalized spreadsheet.

SQL CODE:

The complete SQL code is provided in the attached .sql file.

OUTPUT:

Showing rows 0 - 4 (5 total, Query took 0.0348 seconds.) [OrderID: **O-501... - O-503...**] [BookID: **B-1... - B-3...**]

```
SELECT o.OrderID, o.OrderDate, c.CustID, c.CustName, c.CustEmail, b.BookID, b.BookTitle, b.Publisher, b.UnitPrice, od.Qty FROM
Orders_3NF o JOIN Customers_3NF c ON o.CustID = c.CustID JOIN OrderDetails_3NF od ON o.OrderID = od.OrderID JOIN Books_3NF b ON
od.BookID = b.BookID ORDER BY o.OrderID, b.BookID;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

OrderID ▲ 1	OrderDate	CustID	CustName	CustEmail	BookID	BookTitle	Publisher	UnitPrice	Qty
O-501	2026-04-02	C-11	Bilal	bilal@x.com	B-1	SQL Basics	Pearson	1200.00	1
O-501	2026-04-02	C-11	Bilal	bilal@x.com	B-2	Python 101	OReilly	1500.00	2
O-502	2026-04-03	C-12	Areeba	areeba@x.com	B-1	SQL Basics	Pearson	1200.00	3
O-503	2026-04-05	C-11	Bilal	bilal@x.com	B-2	Python 101	OReilly	1500.00	1
O-503	2026-04-05	C-11	Bilal	bilal@x.com	B-3	Networks	Pearson	1800.00	1

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

QUERY 2: Customer Total Spend

This query calculates total spending per customer by multiplying quantity by unit price and summing across all orders.

SQL CODE:

The complete SQL code is provided in the attached .sql file.

OUTPUT:

Showing rows 0 - 1 (2 total, Query took 0.0048 seconds.)

```
SELECT c.CustID, c.CustName, SUM(od.Qty * b.UnitPrice) AS TotalSpend FROM Customers_3NF c JOIN Orders_3NF o ON c.CustID = o.CustID JOIN
OrderDetails_3NF od ON o.OrderID = od.OrderID JOIN Books_3NF b ON od.BookID = b.BookID GROUP BY c.CustID, c.CustName ORDER BY TotalSpend
DESC;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

CustID	CustName	TotalSpend ▼ 1
C-11	Bilal	7500.00
C-12	Areeba	3600.00

Show all | Number of rows: 25 Filter rows: Search this table

VERIFICATION:

The results match the original data:
• Bilal (C-11) spent: (1200×1) + (1500×2) + (1800×1) + (1500×1) = 7500
• Areeba (C-12) spent: (1200×3) = 3600

This confirms that the 3NF decomposition preserved all data correctly and relationships between tables are properly maintained through foreign keys.

TASK 6: Reflection

The 3NF schema prevents insertion anomalies because new books or customers can be added without requiring an order. The update anomaly is eliminated since each fact is stored only once; if a publisher changes its name, only one row needs updating. The deletion anomaly is resolved because deleting an order only removes order records, not the book information. Overall, normalization through 1NF, 2NF, and 3NF eliminates data redundancy, prevents anomalies, and maintains data integrity through proper primary and foreign keys.